DeFuzz: Invariant-Guided Testing & Self-Healing Framework for Compiler Defenses

IEEE S&P / CCS Style | GCC 13-17 & LLVM 15-19 | Agentic Directed Fuzz

PHASE 1: Invariant SpecGen — CoT + RAG

Knowledge Corpus
- Compiler Sources & Comments
- Architecture ABI & ISA Specs
- Historical CVE Vulnerabilities

Extraction & Synthesis
Path A: Segmented CoT
Extracts assets, bounds & guarantees

Path B: Homologue RAG
Cross-backend root-cause transfer

Security Invariant Schema
```
id: "INV-SP-042" (Stack-Protector)
asset: "Saved Return Address & Frame"
precond: "VLA / Dynamic alloca present"
guarantee: "canary_offset > max_buf"
observation: "Guard intact on overflow"
budget: Fixed Window T (e.g. 30m)
verdict: {PASS, FAIL, NA, ERROR}
```

PHASE 2: Source Mapping & Log — Free Agent

Source-Analysis Agent (Claude Code / Free Agent)
- **CodeQL Engine** — AST & Dataflow
- **IPA CallGraph** — -fdump-ipa-cgraph
- **File / Grep MCP** — Source Context

3-Level Hierarchical Code Mapping
L1: Defense Pass Scope
`pass_prologue_and_epilogue (aarch64.cc)`

L2: Function Call-Graph Cluster
`aarch64_layout_frame → assign_stack_local`

L3: Structs & Critical Conditions
`struct aarch64_frame, locals_offset < canary`

ID-Prefixed Log Injection & Rebuild
```
// Injected Unique ID Prefix Logging
fprintf(stderr, "[DEFUZZ-LOG-SP-042] frame: %ld",
frame.canary_offset);
```

Rebuild Target Compilers
GCC 13-17 & LLVM 15-19 with Injected Probes

PHASE 3: ISA Oracles — Dual-Mode

ISA Specialization
- AArch64
- x86_64
- RISC-V (RV64)
- ARM32 / Thumb

Bifurcated Checkers
STATIC — Binary Analyzer
- Inspects ELF symbols & sections
- Disassembly & frame layout

DYNAMIC — Runtime Probe
- Executes test harness & probes
- Register snapshots & canary check

Four-State Verdict Contract
- PASS (Safe)
- FAIL (Buggy!)
- N/A (Skip)
- ERROR (Diag)

Deterministic Evidence Adjudication
LLM never acts as bug judge

PHASE 4: Invariant-Directed Fuzzing Loop — Online Engine

Target Lock & Budget Allocator (Equal Budget T per Invariant)
Locks: [Invariant INV_k] + [ISA Checker Suite] + [Time Budget T = 30m]

Generator Agent (Checker-Aware PBT Synthesizer) — LLM Reasoning
1. Macro Switch Matrix
-D BUF_SZ=... parameters
Compile-time variances
`#ifdef USE_VLA`

2. Anti-Optimization
asm volatile memory
Volatile sink escapes
`__optnone / noinline`

3. Dynamic Probes
argc/argv binary search
Runtime probe hooks
`target_vla_frame()`

Target Instrumented Compiler Execution & Telemetry
GCC 13-17 Instrumented
`-fstack-protector -fstack-usage`
`Emits: [DEFUZZ-LOG-SP-042]`

LLVM 15-19 Instrumented
`-fstack-size-section -Rpass-missed`
`Emits: [DEFUZZ-LOG-LLVM-019]`

Checker Adjudication
Invariant Violated (FAIL)?

- YES [FAIL] → Invariant Violated (Bug Found!)
- NO [PASS/NA/ERR] → Diagnostics & Feedback Agent

LATERAL LOOP

GUIDANCE LOOP

Invariant Violated (Bug Found!)
1. C-Reduce PoC Minimizer
Reduces to minimal triggering C code
Re-validated against same checker

2. Lateral Divergent Mining
Guides Generator to explore more bugs:
- Mutate alternative buffer layouts
- Diverse alloca/VLA nesting depths
- Exploit sister struct alignments
- Continues in same time budget T

→ Maximize bugs in budget T!

3. Verified 0-Day / Patch PoV
Deterministic proof artifact stored
Feeds RAG Probe Pool (Bottom)

Diagnostics & Feedback Agent
1. Advanced Diagnostic MCP
- Stack usage bytes (detect stack=0)
- Optimization missed diagnostics
- Injected Log IDs execution hit rate

2. Adaptive Checker Rewrite
If NA/Error decisions > threshold K:
- Refinement Agent rewrites checker
- Re-verified via Tier 1 Baseline Gate
- Zero goalpost drift verified

→ Hot-reloads patched Checker!

3. Anti-Optimization Guidance
Actionable hints for next round:
- `Add memory clobber barrier`
- `Force volatile escape sink`
- `Adjust frame layout switches`

Adaptive Checker Rewrite (Hot Reload)

Continuous Knowledge Blackboard, Grounding Gates & Historical Bug Probe Pool — Auditable & Replayable

Versioned Storage Store
- **Invariant Queue & Catalog**
  Structured specs: StackProtector, CET/IBT, PAC, CFI, ASan, ShadowStack, Fortify
- **Seed Mutation & Telemetry**
  Branch coverage bitmaps, GIMPLE/RTL dumps, and runtime probe telemetry logs
- **Replay & Ablation Infrastructure**
  Deterministic replay across compiler versions (GCC 13-17 / LLVM 15-19)

Two-Tier Grounding Gates
Tier 1: Canonical Baseline Gate
Replayed on canonical test suites
Guarantees zero false-positive oracle drift

Tier 2: Novelty & Entailment Filter
Novelty score threshold > 85.0 vs. manual rules
Validates logical entailment from source

Hot-Reload Gate for Patched Checkers
Patched checkers must re-pass Tier 1 gate
Enables continuous self-healing without regression

Closed-Loop RAG Probe Corpus
0-Day Homologue Loop
Confirmed root causes distilled into new probes
Searches sibling flaws in other backends!

Homologue Transfer Yields:
- AArch64 VLA bypass → RISC-V Frame
- Temporary register leak → CET IBT
- SRA optimization wipe → CFI check

→ Continuous evolutionary testing
Feeds SpecGen RAG knowledge base

Homologue Probes Fed to SpecGen RAG